Advanced Data Management
Quiz Refresher: Chapters 1–3, Answer Sheet

Answers to the 30 recall questions, the 8 multiple-choice questions and the 6 midterm-style problems

Advanced Data Management

2026-10-27

Answers

Round A

A1. Connection/session manager, query processor, storage manager, transaction manager, catalog.

A2. Parse, bind, rewrite (logical plan), optimise (physical plan), execute.

A3. Binding, by looking the name up in the catalog. The parser checks grammar only.

A4. You state *what* you want, not *how*; the DBMS must choose the plan itself, which is what the optimiser does.

A5. `open`, `next`, `close`.

A6. Tuples flow one at a time up the tree without materialising intermediate results. A sort (and the build side of a hash join) must consume all input before producing any, so it breaks the pipeline.

A7. The most deeply indented line; read the tree bottom-up.

A8. The child under the `Hash` node; the other child is probed.

A9. Moving a filter below a join so it runs on the smaller input first; fewer rows reach every operator above.

A10. OLTP: many small reads and writes of whole rows, row storage. OLAP: few large scans of a few columns over many rows, column storage.

A11. The number of page reads and writes; 8 KB.

A12. A disk page read costs millions of CPU cycles, so pages are where the time goes, and they are easy to count by hand.

A13. It treats random and sequential reads as equal; it ignores caching (Chapter 6); it ignores CPU-heavy work such as in-memory sorting.

A14. B; about $B/2$; B always, because you must read every page to be sure.

A15. $\lfloor (8192 - 24)/(\text{record} + 4) \rfloor = \lfloor 8168/(\text{record} + 4) \rfloor$.

A16. So tuples can move inside the page during compaction without changing their (page, slot) id, which indexes rely on, and to allow variable-length tuples.

A17. Heap B (average $B/2$); sorted $\log_2 B$; hashed about 1.

A18. Adjacent keys hash to unrelated buckets, so a range touches every bucket: cost B.

A19. A random read moves the disk arm (about 10 ms); sequential reads stream. The DBMS lays related rows out contiguously, reads in large runs and prefetches.

A20. The access pattern (a one-pass scan should not evict hot pages) and the required write ordering (log before data, for durability).

A21. A separate sorted (or hashed) structure beside the heap that maps key values to row ids.

A22. The fraction of the table's rows a predicate returns.

A23. An index scan costs about one random page read per matching row; past a few percent of the table, B sequential reads are cheaper, and the optimiser picks the scan.

A24. $h = \lceil \log_f N \rceil$, typically 3–4; a point lookup costs $h + 1$ reads (one more for the heap page).

A25. So a range query descends once and walks the chain of leaves in key order instead of re-searching per key.

A26. The leaf splits into two half-full leaves and the middle key is pushed up to the parent; if the parent is full it splits too, and the tree grows only when the root splits.

A27. Clustered: heap order follows the key, so a range reads adjacent heap pages. Secondary: heap order is unrelated, so each match is a random page read.

A28. A multi-column index on (a, b, c) serves predicates on a leading prefix: a; a, b; a, b, c. It cannot serve b alone, and after a range on a the later columns are only filters.

A29. Partial: the query always has the same selective predicate (`WHERE status = 'active'`). Expression: the query filters on `lower(name)`. Covering: the query needs only indexed columns, so the heap is never touched (index-only scan).

A30. Disk space; write amplification (every insert, update of the key, and delete also touches the index); optimiser effort and more chances for a wrong plan.

Round B

B1. **(d) 4,000.** An absent key forces a read of every page; $B/2$ is the average only for a present unique key.

B2. **(b) 53.** $\lfloor 8168/(150 + 4) \rfloor = \lfloor 53.04 \rfloor = 53$. Forgetting the slot gives 54; forgetting the floor is the other trap.

B3. **(b) 4.** $200^3 = 8{,}000{,}000 \geq 5{,}000{,}000$, so height 3; plus one heap read $= 4$. (d) is the $\log_2$ trap: $\log_2 5 \times 10^6 \approx 23$ is for binary trees, not fanout 200.

B4. **(c).** `city` is not a leading column; the index is sorted by `country` first, so `city` values are scattered. (d) is a range on the leading column, which

the index serves.

B5. **(b) binding.** `ordrs` is a syntactically valid identifier; only the catalog lookup reveals there is no such table.

B6. **(b) about 1/30.** The column store reads only the `price` column's pages; the row store must read every page because every row's `price` sits beside 29 other columns.

B7. **(c).** Hashing destroys order, so ranges, sorts and prefix matches cannot use it; only equality on the whole key.

B8. **(c) up to B.** The row must go in key order, so on average half the file shifts; in the worst case all of it.

Round C

Solution: C1. Storage estimation

(a) Columns $8 + 4 + 12 + 8 + 8 = 40$ bytes; plus the 24-byte header $= 64$, already a multiple of 8. With the 4-byte slot each record costs 68 bytes. Records per page $= \lfloor 8168/68 \rfloor = \lfloor 120.1 \rfloor = 120$. Pages $= \lceil 3{,}000{,}000/120 \rceil = 25{,}000$. Size $= 25{,}000 \times 8$ KB $= 200{,}000$ KB ≈ 200 MB.
(b) Full scan 25,000. Present unique key, average $B/2 = 12{,}500$. Absent key 25,000: every page must be read to prove absence.
(c) 200 MB/500 MB/s $= 0.4$ s.
(d) Record $40 + 100 = 140$, plus header 164, aligned to 8 $\rightarrow$ 168, plus slot 172. Records per page $\lfloor 8168/172 \rfloor = \lfloor 47.5 \rfloor = 47$. Pages $\lceil 3{,}000{,}000/47 \rceil = 63{,}830$, about 511 MB, scan about 1.0 s. The scan got $63{,}830/25{,}000 \approx 2.55\times$ slower. Adding a column is never free: every query that scans pays for it even if it never reads `notes`.
(e) `SELECT pg_relation_size('tickets') / 8192;` for the page count, and `SELECT relpages, reltuples FROM pg_class WHERE relname = 'tickets';` for the planner's view of it (after `ANALYZE`). To see rows per page directly: `SELECT count(*) FROM heap_page_items(get_raw_page('tickets', 0));` from `pageinspect`. A large gap from the estimate means dead tuples (`VACUUM`) or TOAST.

Solution: C2. Choosing a file organisation

(a) Hashed. Equality search costs about 1 page versus $\log_2 B$ sorted and $B/2$ heap; the nightly update is a batch, so the hashed file's weakness (reorganising on growth) is irrelevant.
(b) Heap. Inserts cost about 1 page; a sorted file would cost up to B per insert. Reads are full scans, which cost B in every organisation, so nothing is lost.
(c) Sorted by date. Range search costs $\log_2 B+$ matching pages, versus B for heap and B for hashed, which scatters adjacent dates across buckets.
(d) It keeps the heap and adds an index on `sensor_id`: a B+ tree gives $h + 1 \approx 4$ reads per lookup while inserts stay cheap (heap page plus one

leaf page). Changing the file organisation would trade the write cost for the read cost; an index pays a small write cost for both.

Solution: C3. B+ tree arithmetic

(a) $150^2 = 22{,}500$ and $150^3 = 3{,}375{,}000 \geq 3{,}000{,}000$, so height $= \lceil \log_{150} 3 \times 10^6 \rceil = 3$. A point lookup reads 3 index pages plus 1 heap page $= 4$, against 12,500 on average for a heap scan: about 3,000× cheaper.
(b) Descend 3; leaves $\lceil 2{,}000/150 \rceil = 14$; heap fetches up to 2,000 (one random page per row, since `id` order is unrelated to heap order). Total about $3 + 14 + 2{,}000 \approx 2{,}017$ reads versus 25,000 sequential: the index wins by about 12×.
(c) Descend 3; leaves $300{,}000/150 = 2{,}000$; heap fetches up to 300,000. About 302,000 random reads versus 25,000 sequential reads: the scan wins by about 12× even though it reads 90% "useless" rows. The optimiser picks the sequential scan.
(d) The index scan costs roughly one read per match, the scan costs $B = 25{,}000$. They tie near 25,000 matches, which is $25{,}000/3{,}000{,}000 \approx 0.8\%$ of the table. The true crossover is lower because the index reads are random (each one a seek on a disk, and a cache miss in any case) while the scan's are sequential, so the planner charges each random page several times more than a sequential one. Lab 3, Part 4 finds it experimentally around one or two percent at best.
(e) The leaf (10, 20, 30, 40) is full. Split: (10, 20) and (25, 30, 40); push the first key of the new right leaf, 25, into the parent. Parent becomes (10 | 25 | 50). In a B+ tree 25 stays in the leaf as well, since every key must appear at leaf level.

Solution: C4. Index design

	Index	Type	Why
(a)	`CREATE UNIQUE INDEX ON tickets(buyer_email)`	B-tree	Exact match returning one row: $h + 1 \approx 4$ reads instead of 25,000.
(b)	`CREATE INDEX ON tickets(sold_at)`	B-tree	Ranges need order: descend once, walk the leaves. A hash index cannot do this.
(c)	`CREATE INDEX ON tickets(status) WHERE status = 'refunded'`	partial B-tree	1% of rows, 30,000: index scan about 30,000 reads versus 25,000 scan is borderline, but the partial index is tiny (30,000 entries) and a bitmap heap scan sorts the fetches into page order, so it wins.
(d)	none		See below.
(e)	`CREATE INDEX ON tickets(lower(buyer_email))`	expression B-tree	The index on `buyer_email` sorts by the raw value; `lower()` is a different key.
(f)	`CREATE INDEX ON tickets(event_id, sold_at)`	multi-column B-tree	Equality on the leading column, then a range on the second: within one `event_id` run the `sold_at` values are contiguous. The reverse order would not serve the equality.
(g)	the index of (b)	B-tree	Walk the leaf chain backwards from the right end, stop after 10: about $h + 1$ pages, no sort.

The pattern no index helps: (d). `status = 'sold'` has selectivity 0.95, so $0.95 \times 3{,}000{,}000 = 2{,}850{,}000$ rows. An index scan would cost up to 2.85 million random page reads on a heap of only 25,000 pages: each page re-read about 114 times. The sequential scan reads 25,000 pages once. No index can beat that; the planner will scan.

Solution: C5. Reading a plan

(a) The deepest node: `Seq Scan on events` with its filter. Its output is hashed in memory (the build side); then `Seq Scan on tickets` is probed against the hash table.

(b) Predicate pushdown, done in the rewrite stage (logical plan) and confirmed by the optimiser. The filter mentions only `events` columns, so it can run before the join; it shrinks the build side from all events to the Moroccan ones, and every operator above sees fewer rows.

(c) `Seq Scan on events`: estimated 4, actual 9,000, off by 2,250×. The error propagates: the join estimate 120 versus actual 410,000. Cause: stale statistics (the table grew or the `country` distribution changed since the last `ANALYZE`). Fix: `ANALYZE events;` then re-run `EXPLAIN ANALYZE`.

(d) Binding. `countr` is grammatically a valid identifier, so parsing succeeds; the binder looks it up in the catalog, finds no such column on `events`, and rejects the query.

(e) The transaction manager, through its lock manager: a lost update is a concurrency anomaly, not a storage or query-processing fault.

Solution: C6. Discussion

(a) Three reasons, each with its failure under the OS alone.

1. **Access-pattern knowledge.** The DBMS knows a sequential scan is one-pass and can protect hot pages (index roots, small dimension tables) from being evicted by it. The OS sees anonymous reads and would flush the useful pages (sequential flooding), so the next lookups miss.
2. **Write ordering for durability.** Recovery requires the log record to reach disk before the data page it describes (write-ahead logging). The OS flushes cached pages in whatever order it likes; a crash between the two writes leaves a data page on disk with no log record to undo it, and recovery is impossible.
3. **Control over eviction and pinning.** An operator working on a page must know it will stay in memory (pinned) until it is done, and the DBMS wants to choose the victim (LRU variants, clock) with knowledge of the query plan. The OS can page out anything at any time, and keeping a copy in both caches wastes memory.

(b) $\log_2 N$ is the depth of a *binary* tree. A B+ tree node is a whole page holding hundreds of keys, so the height is $\log_f N$ with fanout f in the hundreds: for $f = 100$, $\lceil \log_{100} 10^6 \rceil = 3$. Add one heap read: about 4 page reads, not 20. Each page read eliminates a factor of not 2.

Summary: If you missed these, reread

- A3, A5–A9, B5, C5: Chapter 1, Section 3 (the life of a query) and the `EXPLAIN` technique box.
- A11–A20, B1, B2, B8, C1, C2, C6(a): Chapter 2, Sections 2–4 and 6.2.
- A21–A30, B3, B4, B6, B7, C3, C4, C6(b): Chapter 3, Sections 2–5.